PROJECT MANAGEMENT SYSTEM

22ITP61-PROJECT REPORT
Submitted by

YAZHINI T
(23ITR179)

RITHIK VALAYALKARANPUDHUR SUBRAMANIYAN
(23ITR131)

in partial fulfilment of the

requirements for the award of the

degree of

BACHELOR OF TECHNOLOGY

IN

INFORMATION TECHNOLOGY

DEPARTMENT OF INFORMATION TECHNOLOGY

KONGU ENGINEERING COLLEGE

(Autonomous)

PERUNDURAI ERODE – 638 060

MARCH 2026

DEPARTMENT OF INFORMATION TECHNOLOGY KONGU ENGINEERING COLLEGE

(Autonomous)

PERUNDURAI ERODE – 638060

MARCH 2026

BONAFIDE CERTIFICATE

This is to certify that the Project report entitled **" PROJECT MANAGEMENT SYSTEM"** is the Bonafide record of project work done by **YAZHINI T(23ITR179)** and **RITHIK VALAYALKARANPUDHUR SUBRAMANIYAN(23ITR131)** in partial fulfilment of the requirements for the award of the Degree of Bachelor of Technology in **INFORMATION TECHNOLOGY** of Anna University Chennai during the year 2025 - 2026.

SUPERVISOR **HEAD OF THE DEPARTMENT**

Date: **(Signature with seal)**

Submitted for the end semester viva voce examination held on _____.

INTERNAL EXAMINER **EXTERNAL EXAMINER**

DEPARTMENT OF INFORMATION TECHNOLOGY

KONGU ENGINEERING COLLEGE

(Autonomous)

PERUNDURAI ERODE – 638060

MARCH 2026

DECLARATION

We affirm that the Project Report titled **" PROJECT MANAGEMENT SYSTEM "** being submitted in partial fulfilment of the requirements for the award of Bachelor of Technology is the original work carried out by us. It has not formed the part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

YAZHINI T
(23ITR179)

RITHIK VALAYALKARANPUDHUR SUBRAMANIYAN
(23ITR131)

I certify that the declaration made by the above candidates is true to the best of my knowledge.

Date: Name and Signature of the Supervisor with seal

ABSTRACT

Project Management System is a web-based application developed to help users plan, organize, and manage projects and their related tasks in an efficient and structured manner. In many organizations and academic environments, handling multiple projects manually can be difficult and time-consuming. This system provides a digital platform that simplifies project management and helps users maintain project activities in an organized way.

System includes functionalities such as **user registration and login, project creation, task management, and project tracking**. Users can create multiple projects, add tasks under each project, update task details, and monitor the progress of their work. These features help improve task organization and make it easier to manage project activities.

Application is developed using modern **web technologies with frontend and backend integration**, which enables smooth communication between the user interface and the database. Backend processes manage data storage and retrieval, ensuring that project and task information is securely stored and easily accessible when needed.

Overall, this system provides a **simple, user-friendly, and efficient solution for managing projects and tasks**, helping users improve productivity and maintain project information in a centralized platform.

ACKNOWLEDGEMENT

First and foremost, we acknowledge the abundant grace and presence of Almighty throughout different phases of the project and its successful completion.

We wish to express our gratefulness to our beloved Correspondent **Thiru. E. R. K. Krishnan, M.Com.** and all the trust members of Kongu Vellalar Institute of Technology Trust for providing all the necessary facilities to complete the project successfully.

We express our deep sense of gratitude to our beloved Principal **Dr. R. Parameshwaran M.E.,Ph.D.**, for providing us an opportunity to complete the project.

We express our gratitude to **Dr. S. Anandamurugan M.E.,Ph.D.,** Head of the Department, Department of Information Technology for his valuable suggestions.

We are thankful to our Project Coordinators **Dr. E.M.Roopa Devi M.E.,Ph.D.,** , **Ms.S.Sujitha M.Tech.,** and **Ms.G.Sasikala M.E.,** for their valuable guidance and support to complete our project successfully.

We are thankful to the faculty members of the Department of Information Technology for their valuable guidance and support.

We are grateful to **Mr.Prasanth Deventhiran, Software Analyst at Agilysis** for giving me an opportunity to learn new skills and technologies and enhance my knowledge by providing the necessary infrastructure.

(IF YOU ARE NOT ASSIGNED TO ALUMNI MENTOR OR ALUMNI MENTOR DIDN'T RESPOND PLEASE DO ENTER YOUR **CONSULTANCY MENTOR NAME** IN PLACE OF ALUMNI MENTOR NAME)

TABLE OF CONTENTS

CHAPTER No.	TITLE	PAGE No.
	ABSTRACT	iv
	LIST OF FIGURES	viii
	LIST OF ABBREVIATIONS	ix
1	**INTRODUCTION**	**1**
	1.1 OVERVIEW	1
	1.2 OBJECTIVE	1
2	**SYSTEM SPECIFICATION**	**2**
	2.1 HARDWARE REQUIREMENTS	2
	2.2 SOFTWARE REQUIREMENTS	2
3	**SYSTEM DESIGN**	**3**
	3.1 MODULES DESCRIPTION	**3**
	3.1.1 User Authentication Module	3
	3.1.2 Project Management Module	3
	3.1.3 Task Management Module	4
	3.1.4 Project and Task Viewing Module	4
	3.1.5 API and Backend Processing Module	4
	3.1.6 Database Management Module	5
	3.1.7 User Interface Module	6
		6

4	**RESULTS**	**7**
5	**CONCLUSION**	**8**
	APPENDIX 1- CODING	**9**
	REFERENCES	**26**

LIST OF FIGURES

FIGURE No.	FIGURE NAME	PAGE No.
1.1	LOGIN PAGE	3
1.2	CREATE PROJECT PAGE	4
1.3	EDIT TASK PAGE	4
1.4	PROJECT AND TASK VIEWING PAGE	5
1.5	APIs USED	5
1.6	BACKEND PAGE	6
1.7	DASHBOARD PAGE	6

LIST OF ABBREVIATIONS

Abbreviation	Full Form
CSS	Cascading Style Sheets
C	C Programming Language
HTML	HyperText Markup Language
ID	Identifier
JS	JavaScript
JSX	JavaScript XML
QR	Quick Response (Code)
URL	Uniform Resource Locator
AI	Artificial Intelligence
API	Application Programming Interface
HTTP	HyperText Transfer Protocol
AJAX	Asynchronous JavaScript and XML
I/O	Input / Output
JSON	JavaScript Object Notation
JWT	JSON Web Token

CHAPTER 1

INTRODUCTION

1.1 OVERVIEW

Project Management System is a web-based application designed to help users plan, organize, and manage projects and their related tasks efficiently. The system allows users to securely register and log in, create and manage projects, add tasks to each project, and track the progress of those tasks. It provides a centralized platform where all project activities can be viewed and updated in an organized manner. By storing project and task information in a database, the system ensures proper data management and easy access to project details whenever needed. This application helps improve productivity, simplifies project tracking, and provides a user-friendly interface for managing multiple projects effectively.

1.2 OBJECTIVE

- To develop a web-based system that helps users create and manage projects and tasks efficiently.
- To provide a secure platform for user authentication and data management.
- To help users track project progress and organize their work in a structured manner.

CHAPTER 2

SYSTEM SPECIFICATION

2.1 HARDWARE REQUIREMENTS

Processor	:	Intel Pentium
Processor Speed	:	250MHz to 667MHz
RAM	:	4GB RAM
Hard Disk	:	256GB
Keyboard	:	Standard 104 enhanced
Mouse	:	Local PS/2

2.2 SOFTWARE REQUIREMENTS

Platform	:	Visual Studio Code
Server-Side Script	:	NodeJS
Operating System	:	Windows, macOS, or Linux
Backend	:	MongoDB
Library	:	React Js

CHAPTER 3

SYSTEM DESIGN

3.1 MODULES DESCRIPTION

3.1.1 User Authentication Module

User Authentication Module handles user registration and login functionalities. Users can create a new account using the signup page and access the system through the login page. The module verifies credentials and manages authentication using secure tokens. It ensures that only authorized users can access project and task data.

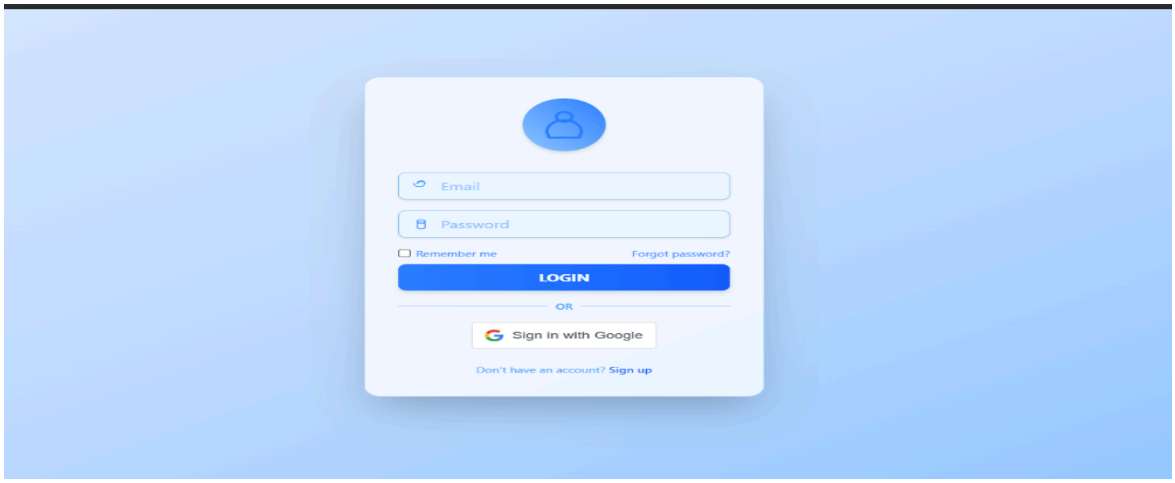

Fig. 1.1 Login Page

3.1.2 Project Management Module

Project Management Module allows users to create, update, view, and delete projects. Each project contains important information such as the project title, description, and associated tasks. It helps users organize work into structured projects for better tracking and management.

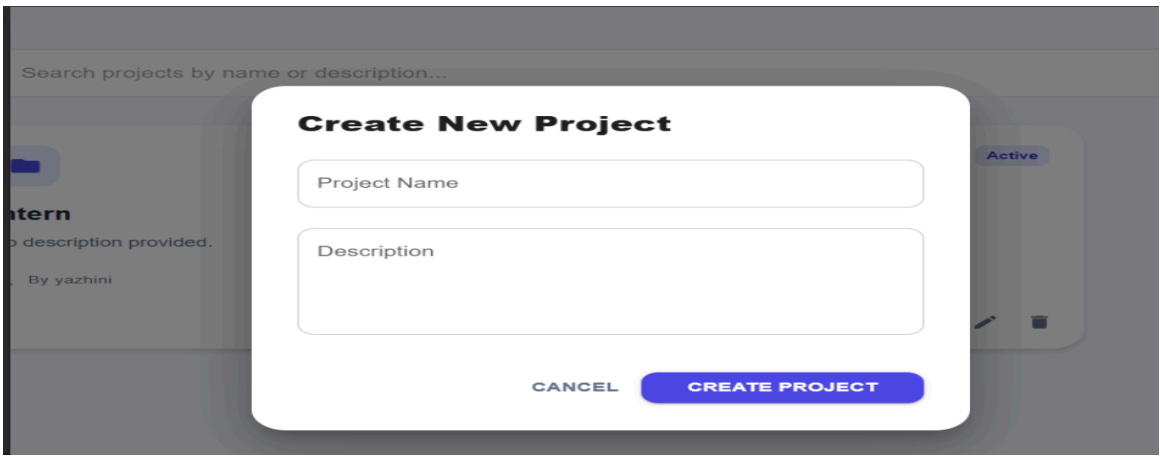

Fig. 1.2 Create Project Page

3.1.3 Task Management Module

Task management module enables users to add, update, and delete tasks within a project. Each task can contain details such as task name, status, and description. This module helps users track progress and manage daily activities related to the project.

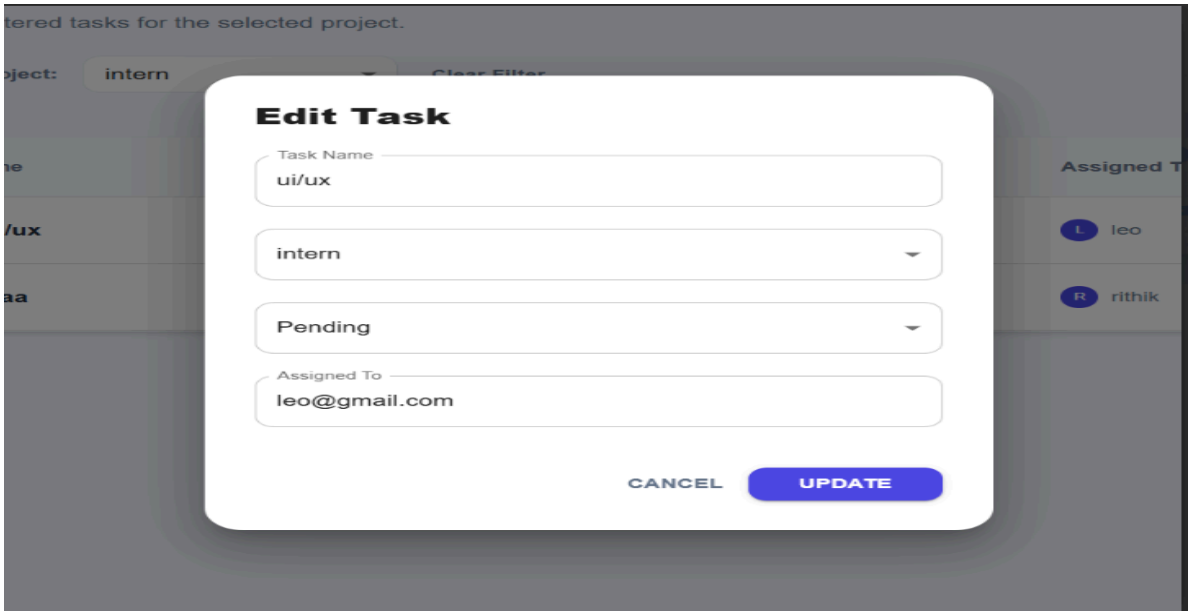

Fig. 1.3 Edit Task Page

3.1.4 Project and Task Viewing Module

Project and Task Viewing Module provides an interface where users can view all projects and their corresponding tasks. It displays project details in an organized format, making it easier for users to monitor progress and manage work efficiently.

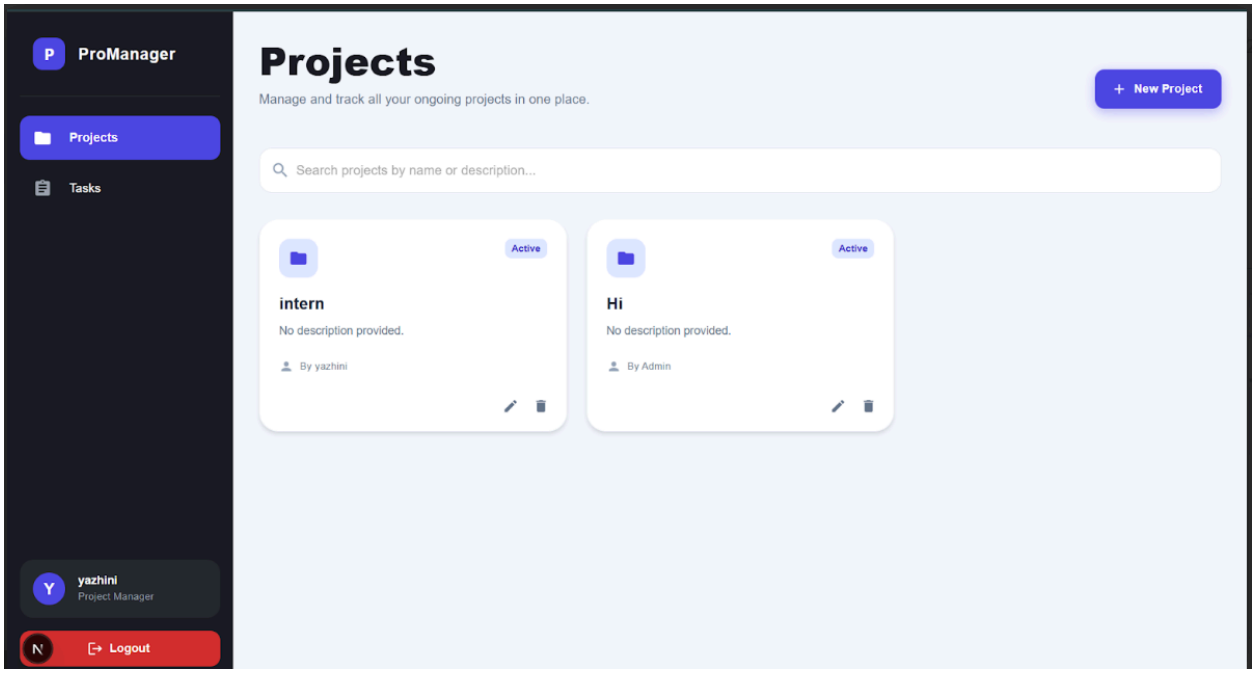

Fig. 1.4 Project and Task Viewing Page

3.1.5 API and Backend Processing Module

Backend Module module manages the communication between the frontend and database. It processes requests such as login, project creation, and task updates through REST APIs. Controllers handle business logic, while routes manage API endpoints.

1. `auth.ts` - Authentication endpoints:
   - `POST /signup`
   - `POST /login`
   - `POST /google` (OAuth login)
   - `GET /users`
2. `project.ts` - Project management endpoints:
   - `POST /` (create)
   - `GET /` (list all)
   - `GET /:id` (get by ID)
   - `PUT /:id` (update)
   - `DELETE /:id` (delete)
3. `task.ts` - Task management endpoints:
   - `GET /` (list all)
   - `GET /:id` (get by ID)
   - `POST /` (create)
   - `PUT /:id` (update)

Fig. 1.5 APIs used

3.1.6 Database Management Module

Database Management module handles data storage and retrieval using database models such as User, Project, and Task. It ensures that all user information, project details, and task records are stored securely and can be accessed whenever required.

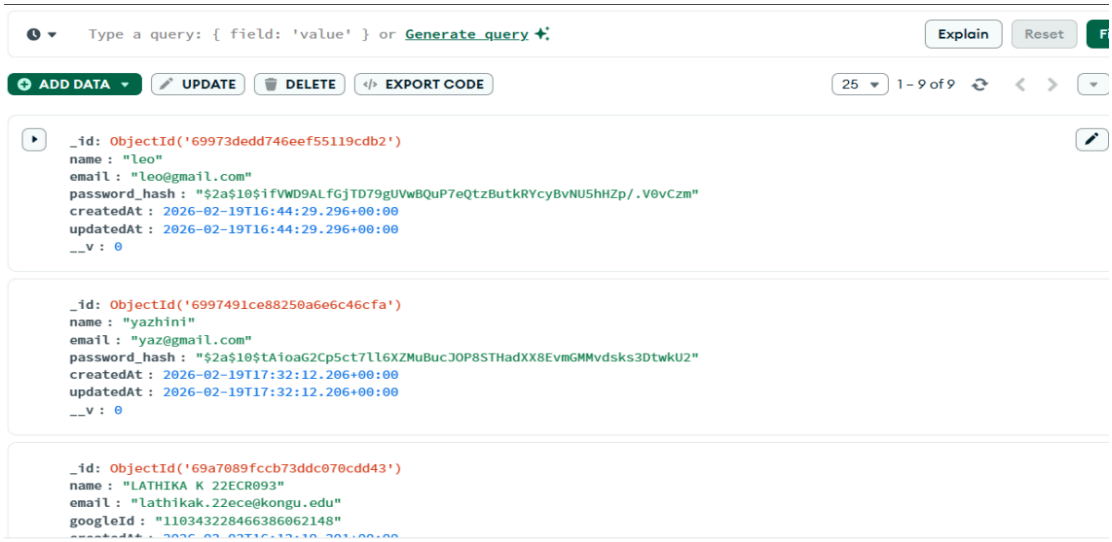

Fig. 1.6 Backend Page

3.1.7 User Interface Module

User Interface module provides the frontend interface developed using Next.js. It includes pages such as login, signup, project list, and task management. The interface allows users to interact with the system easily and perform project management operations.

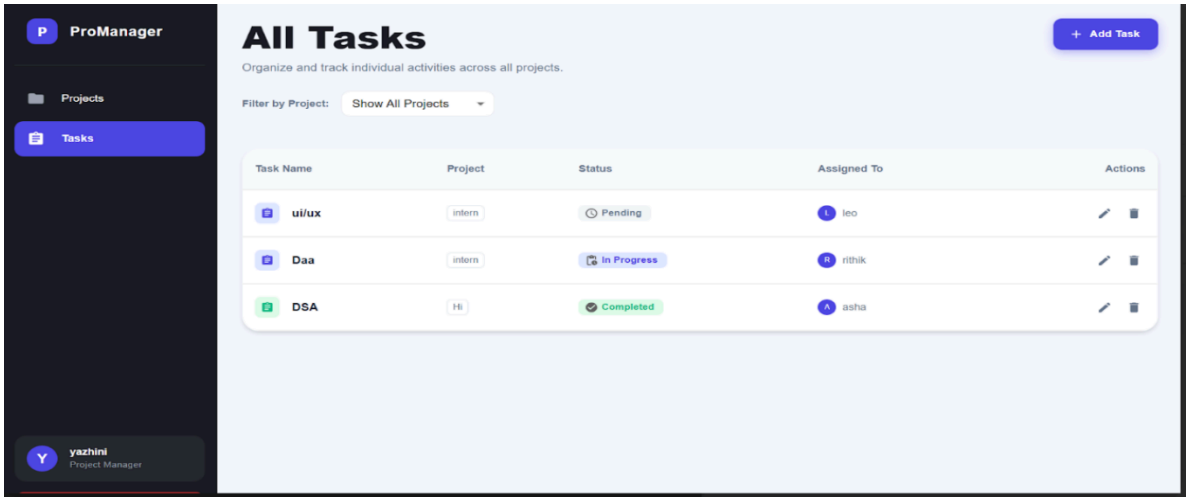

Fig. 1.7 Dashboard Page

CHAPTER 4

RESULTS

Project Management System was successfully designed and implemented as a web-based application that helps users manage projects and tasks effectively. The system provides functionalities such as user registration and login, project creation, task management, and project tracking, allowing users to create multiple projects and add tasks under each project to organize work in a structured manner. The application provides a simple and user-friendly interface that enables users to interact with the system easily, while the backend efficiently manages data storage and retrieval, ensuring that project and task information is securely stored in the database. The integration of frontend and backend technologies enables smooth communication between the user interface and the database. Testing of the system shows that the application successfully performs major operations such as adding, updating, viewing, and deleting projects and tasks, thereby improving productivity by providing a centralized platform for managing and monitoring project activities.

CHAPTER 5

CONCLUSION AND FUTURE WORK

Project Management System developed in this project provides an effective solution for organizing and managing projects and tasks in a digital environment. The system enables users to register, log in, create projects, and manage tasks in a structured and efficient manner. By integrating frontend and backend technologies, the application ensures smooth communication between the user interface and the database. The system also provides a simple and user-friendly interface that allows users to easily track project progress and update task details whenever required. Overall, the project demonstrates the practical implementation of web technologies in building a useful application that improves productivity, simplifies task management, and maintains project information in a centralized and organized manner.

Although the current system provides the basic functionalities required for project management, several enhancements can be made in the future to improve its capabilities. The system can be extended to support **team collaboration**, where multiple users can work on the same project and assign tasks to different members. Additional features such as **notifications and reminders** can be implemented to inform users about upcoming deadlines or task updates. The system can also include **file sharing and document management** to allow users to upload and manage project-related files. Furthermore, **data visualization and reporting tools** can be added to analyze project progress and performance. In the future, the application can also be developed as a **mobile-friendly or mobile application** to provide better accessibility and convenience for users.

APPENDIX 1

CODING

App.tsx :

```
import type { AppProps } from 'next/app'
import { GoogleOAuthProvider } from '@react-oauth/google'
import '@/styles/globals.css'


const GOOGLE_CLIENT_ID = process.env.NEXT_PUBLIC_GOOGLE_CLIENT_ID || '';


export default function App({ Component, pageProps }: AppProps) {
  return (
    <GoogleOAuthProvider clientId={GOOGLE_CLIENT_ID}>
      <Component {...pageProps} />
    </GoogleOAuthProvider>
  )
}
```

Index.tsx:

```
import { useEffect } from 'react';
import { useRouter } from 'next/router';

export default function Home() {
  const router = useRouter();
  useEffect(() => {
    router.replace('/login');
  }, [router]);
  return null;
}
```

login.tsx:

```
import React, { useState, useEffect } from 'react';
import Link from 'next/link';
import { useRouter } from 'next/router';
import { GoogleLogin } from '@react-oauth/google';

import api from '../utils/api';

export default function Login() {
  const router = useRouter();
  const [email, setEmail] = useState((router.query.email as string) || '');
  const [password, setPassword] = useState('');
  const [error, setError] = useState('');
  const [loading, setLoading] = useState(false);
```

```
useEffect(() => {
  const token = localStorage.getItem('token');
  if (token) {
    router.replace('/projects');
  }
}, [router]);

const handleLogin = async (e: React.FormEvent) => {
  e.preventDefault();
  setLoading(true);
  setError('');

  try {
    const res = await api.post('/api/auth/login', { email, password });
    const data = res.data;

    localStorage.setItem('token', data.token);
    router.push('/projects');
  } catch (err: any) {
    setError(err.response?.data?.message || err.message || 'Login failed');
  } finally {
    setLoading(false);
  }
};

const handleGoogleSuccess = async (credentialResponse: any) => {
  setLoading(true);
  setError('');

  try {
    const res = await api.post('/api/auth/google', {
      token: credentialResponse.credential,
    });
    const data = res.data;

    localStorage.setItem('token', data.token);
    router.push('/projects');
  } catch (err: any) {
    setError(err.response?.data?.message || err.message || 'Google login failed');
  } finally {
    setLoading(false);
  }
};

const handleGoogleError = () => {
  setError('Google login failed');
};

return (
  <div className="min-h-screen flex items-center justify-center bg-gradient-to-br
```

```
from-blue-100 via-blue-200 to-blue-300 animate-gradient-x">
    <div className="w-full max-w-sm p-8 bg-white/80 backdrop-blur-md rounded-2xl
shadow-2xl border border-blue-200 flex flex-col items-center transition-transform
duration-300 hover:scale-105">
      <div className="flex flex-col items-center mb-6">
        <div className="bg-gradient-to-tr from-blue-300 to-blue-500 rounded-full p-4 mb-2
shadow-md animate-bounce-slow">
          <svg xmlns="http://www.w3.org/2000/svg" fill="none" viewBox="0 0 24 24"
strokeWidth={1.5} stroke="currentColor" className="w-12 h-12 text-blue-500">
            <path strokeLinecap="round" strokeLinejoin="round" d="M15.75 6a3.75 3.75 0
11-7.5 0 3.75 3.75 0 017.5 0zM4.5 20.25a8.25 8.25 0 1115 0v.75a.75.75 0
01-.75.75h-13.5a.75.75 0 01-.75-.75v-.75z" />
          </svg>
        </div>
      </div>
      <form onSubmit={handleLogin} className="w-full space-y-4">
        {router.query.email && (
          <div className="mb-2 p-2 bg-green-100 border border-green-300 rounded
text-green-700 text-sm text-center animate-pulse">
            Registered successfully! Please login with <b>{router.query.email}</b>.
          </div>
        )}
        {error && (
          <div className="mb-2 p-2 bg-red-100 border border-red-300 rounded text-red-700
text-sm text-center animate-pulse">{error}</div>
        )}
        <div className="relative">
          <span className="absolute left-3 top-1/2 -translate-y-1/2 text-blue-400">
            <svg xmlns="http://www.w3.org/2000/svg" fill="none" viewBox="0 0 24 24"
strokeWidth={1.5} stroke="currentColor" className="w-5 h-5">
              <path strokeLinecap="round" strokeLinejoin="round" d="M16.5 7.5v-.75A4.5 4.5 0
008 6.75v.75m8.25 0A4.5 4.5 0 0012 2.25a4.5 4.5 0 00-4.5 4.5v.75m8.25 0v.75a4.5 4.5 0
01-4.5 4.5h-3a4.5 4.5 0 01-4.5-4.5v-.75" />
            </svg>
          </span>
          <input
            type="email"
            value={email}
            onChange={(e) => setEmail(e.target.value)}
            className="w-full pl-10 pr-4 py-2 border border-blue-300 rounded-lg
focus:outline-none focus:ring-2 focus:ring-blue-400 bg-blue-50 placeholder-blue-300
shadow-sm transition-all duration-200 focus:scale-[1.03]"
            placeholder="Email"
            required
          />
        </div>
        <div className="relative">
          <span className="absolute left-3 top-1/2 -translate-y-1/2 text-blue-400">
            <svg xmlns="http://www.w3.org/2000/svg" fill="none" viewBox="0 0 24 24"
strokeWidth={1.5} stroke="currentColor" className="w-5 h-5">
```

```
          <path strokeLinecap="round" strokeLinejoin="round" d="M16.5 10.5V7.125A2.625
2.625 0 0013.875 4.5h-3.75A2.625 2.625 0 007.5 7.125V10.5m9 0v7.125A2.625 2.625 0
0113.875 20.25h-3.75A2.625 2.625 0 017.5 17.625V10.5m9 0a2.625 2.625 0
00-2.625-2.625h-3.75A2.625 2.625 0 007.5 10.5m9 0H7.5" />
        </svg>
      </span>
      <input
        type="password"
        value={password}
        onChange={(e) => setPassword(e.target.value)}
        className="w-full pl-10 pr-4 py-2 border border-blue-300 rounded-lg
focus:outline-none focus:ring-2 focus:ring-blue-400 bg-blue-50 placeholder-blue-300
shadow-sm transition-all duration-200 focus:scale-[1.03]"
        placeholder="Password"
        required
      />
    </div>
    <div className="flex items-center justify-between text-xs text-blue-500 mb-2">
      <label className="flex items-center">
        <input type="checkbox" className="mr-1 accent-blue-400" /> Remember me
      </label>
      <a href="#" className="hover:underline">Forgot password?</a>
    </div>
    <button
      type="submit"
      disabled={loading}
      className="w-full py-2 bg-gradient-to-r from-blue-500 to-blue-600
hover:from-blue-600 hover:to-blue-700 text-white font-bold rounded-lg shadow-md
transition-all duration-200 hover:scale-105"
    >
      {loading ? 'Logging in...' : 'LOGIN'}
    </button>
  </form>

  <div className="my-4 flex items-center w-full">
    <div className="flex-1 border-t border-blue-300"></div>
    <div className="px-2 text-blue-400 text-xs font-medium">OR</div>
    <div className="flex-1 border-t border-blue-300"></div>
  </div>

  <div className="w-full flex justify-center">
    <GoogleLogin
      onSuccess={handleGoogleSuccess}
      onError={handleGoogleError}
    />
  </div>

  <div className="mt-6 text-center text-blue-400 text-xs">
    Don't have an account?{' '}
    <Link href="/signup" className="text-blue-600 hover:underline font-medium">Sign
```

```
up</Link>
      </div>
    </div>
   </div>
  );
}
```

projects.tsx :

```
import React, { useEffect, useState } from 'react';
import { useRouter } from 'next/router';
import { jwtDecode } from 'jwt-decode';
import api from '@/utils/api';
import Layout from '@/components/Layout';
import {
  Box,
  Typography,
  TextField,
  Button,
  Card,
  CardContent,
  Chip,
  IconButton,
  InputAdornment,
  Grid,
  Dialog,
  DialogTitle,
  DialogContent,
  DialogActions,
  CircularProgress,
  Tooltip,
  Paper,
  GlobalStyles
} from '@mui/material';
import SearchIcon from '@mui/icons-material/Search';
import AddIcon from '@mui/icons-material/Add';
import EditIcon from '@mui/icons-material/Edit';
import DeleteIcon from '@mui/icons-material/Delete';
import FolderIcon from '@mui/icons-material/Folder';
import PersonIcon from '@mui/icons-material/Person';

interface Project {
  project_id: string | number;
  project_name: string;
  description: string;
  created_by: string | number;
  status?: string;
  created_by_name?: string;
}
```

```
export default function ProjectsPage() {
  const router = useRouter();
  const [projects, setProjects] = useState<Project[]>([]);
  const [name, setName] = useState('');
  const [description, setDescription] = useState('');
  const [editingId, setEditingId] = useState<string | number | null>(null);
  const [loading, setLoading] = useState(false);
  const [search, setSearch] = useState('');
  const [showDeleteId, setShowDeleteId] = useState<string | number | null>(null);
  const [openCreateDialog, setOpenCreateDialog] = useState(false);
  const [toast, setToast] = useState<{ type: 'success' | 'error'; message: string } | null>(null);

  const fetchProjects = async () => {
    setLoading(true);
    try {
      const res = await api.get('/api/projects');
      let data = res.data;
      const normalized: Project[] = (Array.isArray(data) ? data : [])
        .map((p: any) => {
          if (Array.isArray(p)) {
            return {
              project_id: p[0],
              project_name: p[1],
              description: p[2],
              created_by_name: p[3],
              created_by: p[4]
            };
          }
          return {
            project_id: p.project_id || p.PROJECT_ID || p.id || p.ID || p._id,
            project_name: p.project_name || p.PROJECT_NAME || p.name || p.NAME || '',
            description: p.description || p.DESCRIPTION || '',
            created_by_name: p.created_by_name || p.CREATED_BY_NAME || '',
            created_by: p.created_by || p.CREATED_BY || p.user_id
          };
        })
        .filter(p => p.project_id !== undefined && p.project_id !== null);

      setProjects(normalized);
    } catch (err) {
      showToast('error', 'Failed to fetch projects');
    }
    setLoading(false);
  };

  useEffect(() => {
    fetchProjects();
  }, []);

  const showToast = (type: 'success' | 'error', message: string) => {
```

```
  setToast({ type, message });
  setTimeout(() => setToast(null), 3000);
};

const handleCreate = async () => {
  if (!name.trim()) return showToast('error', 'Project name is required');
  const token = localStorage.getItem('token');
  let created_by = undefined;
  if (token) {
    try {
      const decoded: any = jwtDecode(token);
      created_by = decoded.id;
    } catch (e) {
      console.error("Failed to decode token", e);
    }
  }
  try {
    await api.post('/api/projects', { project_name: name, description, created_by });
    setName('');
    setDescription('');
    setOpenCreateDialog(false);
    fetchProjects();
    showToast('success', 'Project created successfully');
  } catch (err) {
    showToast('error', 'Failed to create project');
  }
};

const handleUpdate = async (id: string | number) => {
  try {
    await api.put(`/api/projects/${id}`, { project_name: name, description });
    setEditingId(null);
    setName('');
    setDescription('');
    setOpenCreateDialog(false);
    fetchProjects();
    showToast('success', 'Project updated successfully');
  } catch (err) {
    showToast('error', 'Failed to update project');
  }
};

const handleDelete = async (id: string | number) => {
  try {
    await api.delete(`/api/projects/${id}`);
    fetchProjects();
    showToast('success', 'Project deleted');
  } catch (err) {
    showToast('error', 'Failed to delete project');
  }
  setShowDeleteId(null);
```

```
  };

  const startEdit = (project: Project) => {
    setEditingId(project.project_id);
    setName(project.project_name);
    setDescription(project.description);
    setOpenCreateDialog(true);
  };

  const filteredProjects = projects.filter(p =>
    (p.project_name && p.project_name.toLowerCase().includes(search.toLowerCase())) ||
    (p.description && p.description.toLowerCase().includes(search.toLowerCase()))
  );

  return (
    <Layout>
      <Box sx={{ mb: 6, display: 'flex', justifyContent: 'space-between', alignItems: 'flex-end',
gap: 2 }}>
        <Box>
          <Typography variant="h3" sx={{ fontWeight: 800, color: '#1a1c23', mb: 1 }}>
            Projects
          </Typography>
          <Typography variant="body1" sx={{ color: '#64748b' }}>
            Manage and track all your ongoing projects in one place.
          </Typography>
        </Box>
        <Button
          variant="contained"
          startIcon={<AddIcon />}
          onClick={() => {
            setEditingId(null);
            setName('');
            setDescription('');
            setOpenCreateDialog(true);
          }}
          sx={{
            bgcolor: '#4f46e5',
            borderRadius: '12px',
            px: 3,
            py: 1.5,
            textTransform: 'none',
            fontWeight: 600,
            boxShadow: '0 4px 14px 0 rgba(79, 70, 229, 0.39)',
            '&:hover': {
              bgcolor: '#4338ca',
              boxShadow: '0 6px 20px rgba(79, 70, 229, 0.23)',
            }
          }}
        >
          New Project
        </Button>
```

```
</Box>

<Box sx={{ mb: 4 }}>
  <TextField
    fullWidth
    placeholder="Search projects by name or description..."
    variant="outlined"
    value={search}
    onChange={(e) => setSearch(e.target.value)}
    InputProps={{
      startAdornment: (
        <InputAdornment position="start">
          <SearchIcon sx={{ color: '#94a3b8' }} />
        </InputAdornment>
      ),
    }}
    sx={{
      '& .MuiOutlinedInput-root': {
        borderRadius: '16px',
        bgcolor: '#fff',
        '& fieldset': { borderColor: '#e2e8f0' },
        '&:hover fieldset': { borderColor: '#cbd5e1' },
        '&.Mui-focused fieldset': { borderColor: '#4f46e5' },
      }
    }}
  />
</Box>

{loading ? (
  <Box sx={{ display: 'flex', justifyContent: 'center', py: 10 }}>
    <CircularProgress sx={{ color: '#4f46e5' }} />
  </Box>
) : filteredProjects.length === 0 ? (
  <Paper
    sx={{
      p: 10,
      textAlign: 'center',
      borderRadius: '24px',
      boxShadow: 'none',
      border: '2px dashed #e2e8f0',
      bgcolor: 'transparent'
    }}
  >
    <FolderIcon sx={{ fontSize: 64, color: '#cbd5e1', mb: 2 }} />
    <Typography variant="h6" sx={{ color: '#64748b', fontWeight: 600 }}>
      {search ? 'No projects match your search.' : 'No projects found.'}
    </Typography>
    <Button
      onClick={() => setOpenCreateDialog(true)}
      sx={{ mt: 2, color: '#4f46e5', fontWeight: 600 }}
    >
```

```
          Create your first project
        </Button>
      </Paper>
    ) : (
      <Grid container spacing={3}>
        {filteredProjects.map((project) => (
          <Grid item xs={12} sm={6} lg={4} key={`project-${project.project_id}`}>
            <Card
              onClick={() => router.push(`/tasks?project_id=${project.project_id}`)}
              sx={{
                borderRadius: '24px',
                height: '100%',
                display: 'flex',
                flexDirection: 'column',
                transition: 'all 0.3s ease',
                boxShadow: '0 4px 6px -1px rgba(0, 0, 0, 0.1), 0 2px 4px -1px rgba(0, 0, 0, 0.06)',
                border: '1px solid #f1f5f9',
                cursor: 'pointer',
                '&:hover': {
                  transform: 'translateY(-8px)',
                  boxShadow: '0 20px 25px -5px rgba(0, 0, 0, 0.1), 0 10px 10px -5px rgba(0, 0, 0,
0.04)',
                  borderColor: '#e2e8f0'
                }
              }}
            >
              <CardContent sx={{ flexGrow: 1, p: 3 }}>
                <Box sx={{ display: 'flex', justifyContent: 'space-between', mb: 2 }}>
                  <Box
                    sx={{
                      width: 48,
                      height: 48,
                      borderRadius: '14px',
                      bgcolor: '#e0e7ff',
                      display: 'flex',
                      alignItems: 'center',
                      justifyContent: 'center',
                      color: '#4f46e5'
                    }}
                  >
                    <FolderIcon />
                  </Box>
                  <Chip
                    label={project.status || 'Active'}
                    size="small"
                    sx={{
                      borderRadius: '8px',
                      fontWeight: 700,
                      fontSize: '12px',
                      bgcolor: project.status === 'Completed' ? '#dcfce7' : '#e0e7ff',
                      color: project.status === 'Completed' ? '#166534' : '#4338ca',
```

```
                border: 'none'
              }}
            />
          </Box>
          <Typography variant="h6" sx={{ fontWeight: 700, mb: 1, color: '#1e293b' }}>
            {project.project_name}
          </Typography>
          <Typography variant="body2" sx={{ color: '#64748b', mb: 3, lineBreak: 'anywhere'
}}>
            {project.description || 'No description provided.'}
          </Typography>

          <Box sx={{ display: 'flex', alignItems: 'center', gap: 1 }}>
            <PersonIcon sx={{ fontSize: 18, color: '#94a3b8' }} />
            <Typography variant="caption" sx={{ color: '#64748b', fontWeight: 500 }}>
              By {project.created_by_name || 'Admin'}
            </Typography>
          </Box>
        </CardContent>
        <Box sx={{ px: 2, pb: 2, display: 'flex', justifyContent: 'flex-end', gap: 1 }}>
          <Tooltip title="Edit Project">
            <IconButton
              onClick={(e) => { e.stopPropagation(); startEdit(project); }}
              size="small"
              sx={{
                color: '#64748b',
                '&:hover': { color: '#4f46e5', bgcolor: '#e0e7ff' }
              }}
            >
              <EditIcon fontSize="small" />
            </IconButton>
          </Tooltip>
          <Tooltip title="Delete Project">
            <IconButton
              onClick={(e) => { e.stopPropagation(); setShowDeleteId(project.project_id); }}
              size="small"
              sx={{
                color: '#64748b',
                '&:hover': { color: '#ef4444', bgcolor: '#fee2e2' }
              }}
            >
              <DeleteIcon fontSize="small" />
            </IconButton>
          </Tooltip>
        </Box>
      </Card>
    </Grid>
  ))}
 </Grid>
)}
```

```
{/* Create/Edit Dialog */}
<Dialog
  open={openCreateDialog}
  onClose={() => setOpenCreateDialog(false)}
  PaperProps={{
    sx: { borderRadius: '24px', p: 1, maxWidth: 500, width: '100%' }
  }}
>
  <DialogTitle sx={{ fontWeight: 800, fontSize: '24px' }}>
    {editingId ? 'Update Project' : 'Create New Project'}
  </DialogTitle>
  <DialogContent sx={{ pt: 1 }}>
    <Box sx={{ display: 'flex', flexDirection: 'column', gap: 3, mt: 1 }}>
      <TextField
        label="Project Name"
        fullWidth
        value={name}
        onChange={(e) => setName(e.target.value)}
        placeholder="e.g. Website Overhaul"
        sx={{ '& .MuiOutlinedInput-root': { borderRadius: '12px' } }}
      />
      <TextField
        label="Description"
        fullWidth
        multiline
        rows={4}
        value={description}
        onChange={(e) => setDescription(e.target.value)}
        placeholder="What is this project about?"
        sx={{ '& .MuiOutlinedInput-root': { borderRadius: '12px' } }}
      />
    </Box>
  </DialogContent>
  <DialogActions sx={{ p: 3 }}>
    <Button
      onClick={() => setOpenCreateDialog(false)}
      sx={{ color: '#64748b', fontWeight: 600 }}
    >
      Cancel
    </Button>
    <Button
      variant="contained"
      onClick={() => editingId ? handleUpdate(editingId) : handleCreate()}
      sx={{
        bgcolor: '#4f46e5',
        borderRadius: '12px',
        px: 4,
        fontWeight: 600,
        '&:hover': { bgcolor: '#4338ca' }
      }}
    >
```

```
        {editingId ? 'Update Project' : 'Create Project'}
      </Button>
    </DialogActions>
  </Dialog>

  {/* Delete Confirmation */}
  <Dialog
    open={showDeleteId !== null}
    onClose={() => setShowDeleteId(null)}
    PaperProps={{ sx: { borderRadius: '20px', p: 1 } }}
  >
    <DialogTitle sx={{ fontWeight: 700 }}>Delete Project?</DialogTitle>
    <DialogContent>
      <Typography sx={{ color: '#64748b' }}>
        Are you sure you want to delete this project? This action cannot be undone and all
associated tasks might be affected.
      </Typography>
    </DialogContent>
    <DialogActions sx={{ p: 2 }}>
      <Button onClick={() => setShowDeleteId(null)} sx={{ color: '#64748b'
}}>Cancel</Button>
      <Button
        variant="contained"
        color="error"
        onClick={() => showDeleteId && handleDelete(showDeleteId)}
        sx={{ borderRadius: '10px', fontWeight: 600 }}
      >
        Delete
      </Button>
    </DialogActions>
  </Dialog>

  {/* Toast */}
  {toast && (
    <Box
      sx={{
        position: 'fixed',
        bottom: 32,
        right: 32,
        zIndex: 2000,
        bgcolor: toast.type === 'success' ? '#10b981' : '#ef4444',
        color: '#fff',
        px: 3,
        py: 1.5,
        borderRadius: '12px',
        fontWeight: 600,
        boxShadow: '0 10px 15px -3px rgba(0, 0, 0, 0.1)',
        animation: 'slideIn 0.3s ease-out'
      }}
    >
      {toast.message}
```

```
      </Box>
     )}

     <GlobalStyles styles={{
      '@keyframes slideIn': {
        from: { transform: 'translateX(100%)', opacity: 0 },
        to: { transform: 'translateX(0)', opacity: 1 },
      },
     }} />
   </Layout>
  );
}
```

projects.tsx:

```
import React, {useState, createRef} from
'react'; import Loader from
'../components/loader'; import { API_URL }
from '@env';
const RegisterScreen = (props) => {

var formBody = [];
for (var key in dataToSend) {
var encodedKey = encodeURIComponent(key);
var encodedValue = encodeURIComponent(dataToSend[key]);
formBody.push(encodedKey + '=' + encodedValue);
}
formBody = formBody.join('&');
console.log(API_URL);
fetch(`${API_URL}/api/user/register`
, { method: 'POST',
body: formBody,
headers: {
//Header Defination
'Content-Type':
'application/x-www-form-urlencoded;charset=UTF-8',
},
})
.then((response) => response.json())
.then((responseJson) => {
setLoading(false);
console.log(responseJson);
if (responseJson.status === 'success') {
setIsRegistraionSuccess(true); console.log(
'Registration Successful. Please Login to proceed'
);
} else {
setErrortext(responseJson.msg);
}
})
</View>
</View>
```

```
{errortext != '' ? (
<Text style={styles.errorTextStyle}>
{errortext}) : null}
```

Signup.tsx

```
import React, { useState } from 'react';
import Link from 'next/link';
import { useRouter } from 'next/router';

import api from '../utils/api';

export default function Signup() {
  const [name, setName] = useState('');
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [error, setError] = useState('');
  const [loading, setLoading] = useState(false);
  const router = useRouter();

  const handleSignup = async (e: React.FormEvent) => {
    e.preventDefault();
    setLoading(true);
    setError('');

    try {
      await api.post('/api/auth/signup', { name, email, password });

      router.push({
        pathname: '/login',
        query: { email }
      });
    } catch (err: any) {
      setError(err.response?.data?.message || err.message || 'Signup failed');
    } finally {
      setLoading(false);
    }
  };

  return (
    <div className="min-h-screen flex items-center justify-center bg-gradient-to-br
from-blue-100 via-blue-200 to-blue-300 animate-gradient-x">
      <div className="w-full max-w-sm p-8 bg-white/80 backdrop-blur-md rounded-2xl
shadow-2xl border border-blue-200 flex flex-col items-center transition-transform
duration-300 hover:scale-105">
        <div className="flex flex-col items-center mb-6">
          <div className="bg-gradient-to-tr from-blue-300 to-blue-500 rounded-full p-4
mb-2 shadow-md animate-bounce-slow">
            <svg xmlns="http://www.w3.org/2000/svg" fill="none" viewBox="0 0 24 24"
strokeWidth={1.5} stroke="currentColor" className="w-12 h-12 text-blue-500">
```

```
                <path strokeLinecap="round" strokeLinejoin="round" d="M15.75 6a3.75 3.75
0 11-7.5 0 3.75 3.75 0 017.5 0zM4.5 20.25a8.25 8.25 0 1115 0v.75a.75.75 0
01-.75.75h-13.5a.75.75 0 01-.75-.75v-.75z" />
              </svg>
            </div>
          </div>
          <form onSubmit={handleSignup} className="w-full space-y-4">
            {error && (
              <div className="mb-2 p-2 bg-red-100 border border-red-300 rounded
text-red-700 text-sm text-center animate-pulse">{error}</div>
            )}
            <div className="relative">
              <span className="absolute left-3 top-1/2 -translate-y-1/2 text-blue-400">
                <svg xmlns="http://www.w3.org/2000/svg" fill="none" viewBox="0 0 24 24"
strokeWidth={1.5} stroke="currentColor" className="w-5 h-5">
                  <path strokeLinecap="round" strokeLinejoin="round" d="M16.5
7.5v-.75A4.5 4.5 0 008 6.75v.75m8.25 0A4.5 4.5 0 0012 2.25a4.5 4.5 0 00-4.5 4.5v.75m8.25
0v.75a4.5 4.5 0 01-4.5 4.5h-3a4.5 4.5 0 01-4.5-4.5v-.75" />
                </svg>
              </span>
              <input
                type="text"
                value={name}
                onChange={(e) => setName(e.target.value)}
                className="w-full pl-10 pr-4 py-2 border border-blue-300 rounded-lg
focus:outline-none focus:ring-2 focus:ring-blue-400 bg-blue-50 placeholder-blue-300
shadow-sm transition-all duration-200 focus:scale-[1.03]"
                placeholder="Name"
                required
              />
            </div>
            <div className="relative">
              <span className="absolute left-3 top-1/2 -translate-y-1/2 text-blue-400">
                <svg xmlns="http://www.w3.org/2000/svg" fill="none" viewBox="0 0 24 24"
strokeWidth={1.5} stroke="currentColor" className="w-5 h-5">
                  <path strokeLinecap="round" strokeLinejoin="round" d="M16.5
7.5v-.75A4.5 4.5 0 008 6.75v.75m8.25 0A4.5 4.5 0 0012 2.25a4.5 4.5 0 00-4.5 4.5v.75m8.25
0v.75a4.5 4.5 0 01-4.5 4.5h-3a4.5 4.5 0 01-4.5-4.5v-.75" />
                </svg>
              </span>
              <input
                type="email"
                value={email}
                onChange={(e) => setEmail(e.target.value)}
                className="w-full pl-10 pr-4 py-2 border border-blue-300 rounded-lg
focus:outline-none focus:ring-2 focus:ring-blue-400 bg-blue-50 placeholder-blue-300
shadow-sm transition-all duration-200 focus:scale-[1.03]"
                placeholder="Email"
                required
              />
```

```
            </div>
            <div className="relative">
              <span className="absolute left-3 top-1/2 -translate-y-1/2 text-blue-400">
                <svg xmlns="http://www.w3.org/2000/svg" fill="none" viewBox="0 0 24 24"
strokeWidth={1.5} stroke="currentColor" className="w-5 h-5">
                  <path strokeLinecap="round" strokeLinejoin="round" d="M16.5
10.5V7.125A2.625 2.625 0 0013.875 4.5h-3.75A2.625 2.625 0 007.5 7.125V10.5m9
0v7.125A2.625 2.625 0 0113.875 20.25h-3.75A2.625 2.625 0 017.5 17.625V10.5m9 0a2.625
2.625 0 00-2.625-2.625h-3.75A2.625 2.625 0 007.5 10.5m9 0H7.5" />
                </svg>
              </span>
              <input
                type="password"
                value={password}
                onChange={(e) => setPassword(e.target.value)}
                className="w-full pl-10 pr-4 py-2 border border-blue-300 rounded-lg
focus:outline-none focus:ring-2 focus:ring-blue-400 bg-blue-50 placeholder-blue-300
shadow-sm transition-all duration-200 focus:scale-[1.03]"
                placeholder="Password"
                required
              />
            </div>
            <button
              type="submit"
              disabled={loading}
              className="w-full py-2 bg-gradient-to-r from-blue-500 to-blue-600
hover:from-blue-600 hover:to-blue-700 text-white font-bold rounded-lg shadow-md
transition-all duration-200 hover:scale-105"
            >
              {loading ? 'Signing up...' : 'SIGNUP'}
            </button>
          </form>
          <div className="mt-6 text-center text-blue-400 text-xs">
            Already have an account?{' '}
            <Link href="/login" className="text-blue-600 hover:underline
font-medium">Login</Link>
          </div>
        </div>
      </div>
    );
}
```

REFERENCES

1. **React Native Documentation** -Meta (Facebook), https://reactnative.dev/docs/
2. **Node.js Documentation** – OpenJS Foundation, https://nodejs.org/en/docs
3. **Express.js Guide** – Express.js Official Site, https://expressjs.com/
4. **MongoDB Manual** – MongoDB Inc., https://www.mongodb.com/docs/manual/

GIT HUB : https://github.com/rithikvs/project_management

DEPLOYMENT : https://project-management-three-self.vercel.app